

3. Skip List – randomlevel

Program :

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
...
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    ...
}
```

Program Analysis

Introduction:

This is a random level generation , that decides node's height

- if it returns `0`, then node gets 1 pointer[LEVEL: 0].*
- if it returns `1`, then node gets 2 pointers[LEVELS: 0, 1].*
- if it returns `2`, then node gets 3 pointers[LEVELS: 0, 1, 2].*
- etc.*

The randomLevel() function is only ever used during the insertion of a new element.

It is never used when searching for an element, and it is never used when deleting an element.

Analysis

```
int randomLevel() {... }
```

First to start with – there is no parameters.

```
int lvl = 0;
```

So, lvl = 0.

```
while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {  
    lvl++;  
}
```

MAX_LEVEL = 3 as,

```
#define MAX_LEVEL 3
```

say 1st, rand() = 42

1st loop run:

while $\left((rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \right.$
 $\left. AND\ lvl = 0 < MAX_LEVEL = 3 [TRUE] \right):$

$lvl = 0 + 1 = 1$

$$2^{nd}, rand() = 998$$

2nd loop run:

$$while \left(\begin{array}{l} (rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \\ AND \text{ } lvl = 1 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$$

$$lvl = 1 + 1 = 2$$

$$3^{rd}, rand() = 8471$$

3rd loop run:

$$while \left(\begin{array}{l} (rand() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[Remainder] \right] [False] \\ AND \text{ } lvl = 2 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$$

Condition fails , hence :

```
return lvl;
```

return , lvl = 2

How srand helps rand to choose random numbers?

```
int main() {  
    // Seed random number generator for random levels  
    srand((unsigned)time(0));  
    ...  
}
```

When main function runs, **srand ((unsigned)time(0))** executes.

Later on, the program calls rand() inside the randomLevel() function:

```
int randomLevel() {  
    int lvl = 0;  
    // 50% probability to increase the level  
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {  
        lvl++;  
    }  
    return lvl;  
}
```

Computers cannot generate truly random numbers easily; they use mathematical formulas to generate sequences of numbers that look random (pseudo – random numbers).

If you use rand() without srand(), the generator will start with a default seed (usually 1). This means that every time we run our program, rand() will produce the exact same sequence of numbers.

Example:

```
#include <stdio.h>  
#include <stdlib.h> // Required for rand()  
  
int main() {  
    printf("Generating 5 random numbers:\n");
```

```

    for (int i = 0; i < 5; i++) {
        // rand() generates a pseudo-random integer
        printf("%d ", rand());
    }

    printf("\n");
    return 0;
}

```

Output of Run 1:

```

Generating 5 random numbers:
1804289383 846930886 1681692777 1714636915 1957747793

```

Output of Run 2:

```

Generating 5 random numbers:
1804289383 846930886 1681692777 1714636915 1957747793

```

Output of Run 3:

```

Generating 5 random numbers:
1804289383 846930886 1681692777 1714636915 1957747793

```

Same sequence of numbers at each output .

But for SRAND example:

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>    // Required for time()

int main() {
    // Seed the random number generator with the current system
    time.
    // You only need to call this ONCE at the start of your program.
    srand(time(NULL));

    printf("Generating 5 TRULY random numbers:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", rand());
    }
}

```

```
}

printf("\n");
return 0;
}
```

Output of Run 1:

```
Generating 5 TRULY random numbers:
142839210 18493021 984920184 48291048 2948194
```

Output of Run 2:

```
Generating 5 TRULY random numbers:
958392019 39481023 11849302 48920184 7583921
```

Here NULL and 0 is same thing and we used :

```
srand((unsigned)time(0));
```

The srand()function, however, specifically requires an unsigned int (which is usually a 32 – bit positive integer).

*If we write **srand(time(NULL));**, we are handing a signed integer to a function that wants an unsigned integer. The compiler will handle the conversion for us automatically, but modern compilers might throw a warning like this:*

Warning: conversion from 'time_t' to 'unsigned int', possible loss of data.

*By adding **(unsigned)**, we are explicitly telling the compiler, "I know I am converting these data types, do it anyway." This is called type casting, and it suppresses that warning.*

In the context of our Skip List:

- *Skip Lists rely on randomness (a coin toss) to decide how many levels a new node should have.*
- *If rand() produced the exact same sequence every time we ran the program, our Skip List would build itself in the exact same shape for a given set of inputs every single time, which defeats the purpose of testing a probabilistic data structure.*

That is:

Output of Run 1:

```
Generating 5 random numbers:  
1804289383 846930886 1681692777 1714636915 1957747793
```

Output of Run 2:

```
Generating 5 random numbers:  
1804289383 846930886 1681692777 1714636915 1957747793
```

Output of Run 3:

```
Generating 5 random numbers:  
1804289383 846930886 1681692777 1714636915 1957747793
```

It defeats the purpose of testing a probabilistic data structure.

By calling srand((unsigned)time(0)), we are setting the starting seed to the current time (the number of seconds since January 1, 1970). Because the time changes every second, the starting seed is different every time we run the program. This guarantees that rand() will produce a different sequence of random numbers each time, causing our Skip List nodes to genuinely have unpredictable heights on every run.

Best Case

1st, rand() = 8471

1st loop run:

$\text{while} \left(\begin{array}{l} (\text{rand}() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[\text{Remainder}] \right] [\text{False}] \\ \text{AND } \text{lvl} = 0 < \text{MAX_LEVEL} = 3 [\text{TRUE}] \end{array} \right):$

Condition fails , hence :

```
return lvl;
```

return , lvl = 0

In the best – case scenario, the very first call to rand()% 2 returns 1.

***The condition (rand()% 2) == 0 immediately evaluates to false,
and the while loop terminates without executing even once.***

The function simply returns 0. This takes constant time.

Time Complexity : O(1).

Worst Case

$$\mathbf{1^{st}, rand() = 42}$$

1st loop run:

$$\mathbf{while \left(\begin{array}{l} (rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \\ AND \textit{lvl} = 0 < MAX_LEVEL = 3 [TRUE] \end{array} \right):}$$

$$\mathbf{\textit{lvl} = 0 + 1 = 1}$$

$$\mathbf{2^{nd}, rand() = 998}$$

2nd loop run:

$$\mathbf{while \left(\begin{array}{l} (rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \\ AND \textit{lvl} = 1 < MAX_LEVEL = 3 [TRUE] \end{array} \right):}$$

$$\mathbf{\textit{lvl} = 1 + 1 = 2}$$

$3^{rd}, rand() = 1042$

2nd loop run:

$while \left(\begin{array}{l} (rand() = 1042 \% 2) = 0 == 0 \left[\frac{1042}{2} = 0[Remainder] \right] [True] \\ AND \text{ } lvl = 2 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

$lvl = 2 + 1 = 3$

Now , either

$4^{rth}, rand() = 8471$

4^{rth} loop run:

$while \left(\begin{array}{l} (rand() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[Remainder] \right] [False] \\ AND \text{ } lvl = 3 < MAX_LEVEL = 3 [False] \end{array} \right):$

Condition fails , hence :

```
return lvl;
```

$return, lvl = 3$

OR,

$$4^{th}, rand() = 8472$$

4th loop run:

$$while \left(\begin{array}{l} (rand() = 8472 \% 2) = 0 == 0 \left[\frac{8472}{2} = 0[Remainder] \right] [True] \\ AND \text{ lvl} = 3 < MAX_LEVEL = 3 [False] \end{array} \right):$$

Condition fails , hence :

```
return lvl;
```

return , lvl = 3

In the absolute worst – case scenario, rand() % 2 repeatedly returns 0, causing the loop to execute until the second condition lvl < MAX_LEVEL forces it to stop. The loop will run exactly MAX_LEVEL times.

(Note: Since MAX_LEVEL is usually a predefined constant , here MAX_LEVEL = 3, in most Skip List implementations – the worst – case is technically bounded by a constant, which means it can be strictly argued as O(1). However, expressing it as O(MAX_LEVEL) is the most accurate way to show what bounds the worst – case performance).

Time Complexity : O(MAX_LEVEL) ≈ O(1).

Average Case

$rand() \% 2 = 0$, when 0 means Head .

$rand() \% 2 = 1$, when 1 means Tail. When Tail comes , then loop condition is False.

0 times, which flip Tails immediately. Probability $= \frac{1}{2} = \left(\frac{1}{2}\right)^{0+1}$.

1 times, which flip Heads then Tails. Probability $= \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} = \left(\frac{1}{2}\right)^{1+1}$.

2 times, which flip Heads, Heads then Tails. Probability $= \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{8}$
 $= \left(\frac{1}{2}\right)^{2+1}$.

3 times, which flip Heads, Heads, Heads then Tails. Probability
 $= \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{16} = \left(\frac{1}{2}\right)^{3+1}$

.

.

i^{th} times, which flip i times Heads and then Tails . Probability $= \left(\frac{1}{2}\right)^{i+1}$

$S = (0 \text{ runs} \cdot \text{prob}) + (1 \text{ run} \cdot \text{prob}) + (2 \text{ run} \cdot \text{prob}) + \dots$

Plugging in our numbers from Step 1, we get:

$$S = 0 \left(\frac{1}{2}\right) + 1 \left(\frac{1}{4}\right) + 2 \left(\frac{1}{8}\right) + 3 \left(\frac{1}{16}\right) + 4 \left(\frac{1}{32}\right) + \dots$$

$$S = 0 + \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots$$

$$\therefore S = \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots$$

Multiplky entire equation by 2 .

$$2S = \frac{2 \times 1}{4} + \frac{2 \times 2}{8} + \frac{2 \times 3}{16} + \frac{2 \times 4}{32} + \frac{2 \times 5}{64} \dots$$

$$\therefore 2S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} + \dots$$

$$2S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} + \dots$$

—

$$S = 0 + \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots$$

— — — — —

$$\therefore (2S - S) = \left(\frac{1}{2} - 0\right) + \left(\frac{2}{4} - \frac{1}{4}\right) + \left(\frac{3}{8} - \frac{2}{8}\right) + \left(\frac{4}{16} - \frac{3}{16}\right) + \left(\frac{5}{32} - \frac{4}{32}\right) + \dots$$

$$\Rightarrow S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots$$

It is infinite series : $S = \sum_{i=1}^{\infty} ar^{i-1} = a + ar + ar^2 + ar^3 + \dots = \frac{a}{1-r}$

$$S = \sum_{i=1}^{\infty} ar^{i-1} = \sum_{i=1}^{\infty} \frac{1}{2} \left(\frac{1}{2}\right)^{i-1} = \frac{1}{2} \times \left(\frac{1}{2}\right)^{1-1=0} + \frac{1}{2} \times \left(\frac{1}{2}\right)^{2-1=1} + \frac{1}{2} \times \left(\frac{1}{2}\right)^{3-1=2} + \dots$$

→ a is very first number in the series, here $\frac{1}{2}$.

→ r is the common ratio, here, $r = \frac{\text{Term } 2}{\text{Term } 1}$, say $\frac{\frac{1}{4}}{\frac{1}{2}} = \frac{1}{4} \times \frac{2}{1} = \frac{1}{2}$

similarly, $r = \frac{\frac{1}{8}}{\frac{1}{4}} = \frac{1}{8} \times \frac{4}{1} = \frac{1}{2}$

Hence, common ratio, $r = \frac{1}{2}$

$$S = \sum_{i=1}^{\infty} ar^{i-1} = a + ar + ar^2 + ar^3 + \dots = \frac{a}{1-r} = \frac{\frac{1}{2}}{1-\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}} = 1$$

i. e. Average Time Complexity = $O(1)$.

<i>Skip List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Random Level</i>	<i>Best Case</i>	$O(1)$
	<i>Worst Case</i>	$O(\text{MAX_LEVEL})$ $= O(c) \approx O(1)$.
	<i>Average Case</i>	$O(1)$

Space Complexity

1. Input Space Complexity: $O(1)$

This measures the memory required to store the inputs provided to the function.

- **Parametric Space Complexity: 0 bytes ($O(1)$)**

The function signature is `int randomLevel()`. It takes absolutely zero parameters. No arguments are being passed into it, so it consumes no memory for parameters.

- **FULL Data Input (Holistic Input Space): $O(1)$**

The function does not take in or process any external data structures (like arrays, linked lists, or trees). The only external data it looks at is `MAX_LEVEL` (which is likely a global constant or a preprocessor macro `#define`) and the internal state of the `rand()` function. Reading a single global constant takes constant, negligible space.

2. Auxiliary Space Complexity: $O(1)$

This measures the temporary, extra memory the algorithm needs to do its work during execution.

- **Local Variables:** The algorithm creates exactly one local variable: `int lvl = 0;`. This consumes a fixed amount of memory (usually 4 bytes in C/C++), regardless of how large the Skip List gets.
- **Execution Stack:** There is no recursion happening here. The function is called once, runs a simple while loop, and finishes. It only takes up a single, small stack frame in memory.
- **Dynamic Allocation:** There is no `malloc` or `new` keyword inside this function. It creates no dynamic memory on the heap.

Because the memory used does not grow or change based on the size of the Skip List or the number of elements being inserted, the auxiliary space is strictly $O(1)$.

Total Space Complexity = Input Space + Auxiliary Space.

1) Total Space Complexity = Parametric Input Space + Auxiliary Space.

$$O(1) + O(1) = O(1).$$

2) Total Space Complexity = Full Data Input Space + Auxiliary Space.

$$O(1) + O(1) = O(1).$$

$\therefore$ in both cases , Total Space Complexity: $O(1)$.

Algorithm

CONSTANT MAX_ Level= 3

1. Function RandomLevel():

2. SET lvl := 0.

// Loop with a 50% chance of continuing, bounded by MAX_LEVEL = 3

3. Repeat while Random() Mod 2 = 0 AND lvl < MAX_LEVEL :

3. a. lvl = lvl + 1.

[END of Loop]

4. Return lvl

5. End of Function RandomLevel()

[END of Algorithm]
